Vendor Failure Forecast

Week 11: model evaluation and decision-support report

Mirza Ali and Debra Senteu · MS Analytics, Saint Louis University · for the Purchasing & Receiving Manager of a synthetic independent seasonal resort · all data are synthetic

The Week 10 version of the app and package is kept unchanged in the folder Week10_frozen. Everything described here is in Week11_Vendor_Risk_Forecast.

Summary

Each Monday the tool estimates, for about 44 vendors, the chance of a material failure next week, turns that into expected dollars, and lists the top k vendors, where k is the week's review capacity (as few as 4 in the busiest weeks, up to 12 in quiet ones). We tested it on rolling 13-week blocks of 2023 to 2025 data, where every decision in this report was made, and read 2026 as a confirmation only.

Question	Answer
Does the weekly list beat what the manager does today?	**Yes.** The list holds 58% of next-week failure cost against 41% for today's on-time and fill-rate rules (+17.6 [+6.9, +28.6] points). In 2026: 55% against 31% (+24.1 [+10.7, +36.2]).
Is it better than simple rules that need no model?	**Not shown.** Ranking vendors by delivery volume alone holds 53% (2026: 50%). Model minus volume is +5.7 [−3.0, +14.0] points, and its interval includes zero. The pre-stated rule for a meaningful improvement is not met.
Are the percentages right?	**On average, yes in 2024 to 2025:** forecast 4.66% against 4.56% observed, calibration slope 1.03, and observed over forecast is consistent with 1.0 in every segment we could test. In 2026 failures ran above forecast (5.57% against 4.44%, slope 0.76) because the data generator shifted; the drift monitor exists for that.
Is the dollar estimate useful?	**Only as an average.** The cost model's typical miss is $1,163 per failing vendor-week against $1,170 for simply using the average past cost. A failure costs between $444 and $3,696 in 80% of cases and the data give no way to say which one will be expensive.
Did we improve the model?	**No candidate passed the rules we set before running them.** We tested 11 model and recalibration variants, one cost-ranking change and one change to the dollar estimate (13 in all), all on development data, and kept the Week 10 model. Two of them came from a team member's separate two-fold screen. We did fix one input mismatch in the cost estimate and added decision-support features to the app (section 7).

Bottom line for the manager. Use the list to decide where to look first; expect it to hold about half of the failure cost next week, not all of it, and do not read a 19% as "will fail". The tool is a ranking aid that clearly beats today's rules, is not proven better than reviewing the highest-volume vendors, and is weakest for protein and dairy suppliers, in peak weeks and for vendors under an open corrective action. All of this was measured on synthetic data, so it shows how the method behaves, not how a real property would perform.

1. How we tested

Data split and why it fits the use (1a)

	Development: 2023 to 2025	2026 holdout
Weeks and scored vendor-weeks	155 weeks, 6,978 vendor-weeks	52 weeks, 2,441 vendor-weeks
Rows where a material failure happened next week	320 (4.59%)	136 (5.57%)
Failing vendor-weeks (unit of the cost model)	323	138
Rolling out-of-fold test rows (2024 to 2025)	4,846 rows, 221 failures	not applicable
Used for	Fitting, every design choice, every comparison	Confirmation only: scored with the frozen model

Why not a random split. The tool is used forward in time every Monday, and one week's record is nearly the next week's, so mixing weeks at random would let the model learn from the future. We replay what the manager would have had instead: eight 13-week test blocks (2024 Q1 to 2025 Q4). Each block is predicted by a model fitted only on rows whose outcome was already known on the block's first Monday. A row for week t has its outcome (a failure in week t+1) known from Monday of week t+2, so the training rows are those with t at most a−2. Eight blocks give eight separate tests instead of one, which is what lets us ask whether a difference is consistent or a single lucky quarter.

A caveat we cannot remove. The 2026 year was already viewed in Week 10, and one design fix (the zero-failure cap for MRO supplies) was made after seeing it. It is therefore not a clean final test. In Week 11 we fitted, tuned and chose everything on 2023 to 2025 only; scripts 20 to 22 never fit on, tune on or select thresholds on 2026, and the pipeline asserts this. We did not regenerate a fresh, unseen year; if the team can run the Week 7 generator, a new year scored once with the frozen model would give the clean final test this design wants.

One boundary detail. The last development row (the week of 22 December 2025) predicts the first week of 2026, so its label, 2 failures out of 320 in the development labels, is a 2026 outcome that sits in the final fit and in the last test block. Refitting without that week changes the 2026 log loss from 0.1944 to 0.1943 and moves no 2026 probability by more than 0.7 percentage points. We left the frozen Week 10 model as it was and disclose the overlap here.

Leakage check: is every input known on Monday? (1b)

Input	Where it comes from	When it is known	Available at decision time?
Delivery volume (log deliveries per week)	Trailing 13-week delivery count / 13, from the weekly vendor snapshot	Monday of the forecast week, counts deliveries through the previous Sunday	Yes
Rejection rate, last 4 weeks	Weekly vendor snapshot (trailing_4wk_rejection_rate); falls back to the 13-week rate when no deliveries in 4 weeks	Monday of the forecast week	Yes
Rejection rate, last 13 weeks	Weekly vendor snapshot (trailing_13wk_rejection_rate)	Monday of the forecast week	Yes
Vendor category	Vendor master table (fixed attribute)	Always	Yes

Input	Where it comes from	When it is known	Available at decision time?
Vendor effect (reliability history)	Learned only from earlier rows (label known two Mondays after the row)	Monday of the forecast week	Yes
Cost model: occupancy and banquet flag	Demand table; used to size the cost of a failure, not to predict whether one happens	Occupancy forecast and booked banquets are known ahead; Week 11 tests using last week's values	Yes
Open corrective action	Corrective-action log	Excluded from the model (a manager action); used only as an eligibility filter for the review list	Yes
Oracle (true reliability) table	Generator internals	Never read	No (never read)

All 10,120 vendor-week records (scored or not) carry an as-of date equal to their own Monday (checked). As a negative control we also measured each input as it would look if it had secretly included the target week: a leaked input should look much better than the as-of version, and none of the inputs we use does.

Input	AUC as of Monday	AUC if it included the target week	Jump
Delivery volume (deliveries per week)	0.758	0.764	+0.006
Rejection rate, last 4 weeks	0.554	0.664	+0.110
Rejection rate, last 13 weeks	0.607	0.698	+0.091
(not used) Failures in last 13 weeks	0.684	0.883	+0.199

The last row is an input we do not use. It shows what a leak looks like: a jump of about 0.2 AUC. The manager's own open corrective action stays out of the model and is used only to decide which vendors are eligible for the list. The generator's hidden reliability table is never read.

One assumption. The cost estimate uses next week's expected occupancy and whether a banquet is booked. We assume those are known on Monday from reservations; they size the dollar cost and never enter the probability or the ranking. A strict version that uses only last week's values, with nothing from the week being predicted, performs the same (typical miss $1,172 against $1,163).

Rules for "a meaningful improvement", set before the Week 11 numbers were produced

KPI	The model must…	Result
KPI 1: failure probability	beat the best simple baseline (category rate or the vendor's own history) on log loss, with a 95% paired interval excluding zero, and in at least 6 of 8 blocks	Not met
KPI 1: ranking	reach an AUC at least 0.03 above the best simple baseline, interval excluding zero	Not met
KPI 2: dollar cost of a failure	have a typical miss at least 5% lower than "use the average past failure cost", interval excluding zero	Not met
Decision: weekly list	hold at least 5 percentage points more next-week failure cost than the strongest simple list, interval excluding zero	Not met

2026 is read as confirmation only: the same direction as in development is "consistent", anything else is "not confirmed". We disclose that these thresholds were written after the Week 10 results on 2026 were known to us, so they are a fair reading, not a blind pre-registration.

2. What the measures mean for the manager (requirement 1c)

Failures happen in about 1 vendor-week in 20, so plain "accuracy" would reward a tool that says nobody will ever fail (about 95% accurate and useless). We use measures that reflect the actual decision: which few vendors to look at.

Measure	In plain words	Why the manager cares	How to read it
Share of failure cost inside the list (decision KPI)	Of the money lost next week to failures at vendors you could still act on, how much belonged to vendors on the weekly list.	It is the whole point: did looking at k vendors cover the expensive trouble?	A random list of that size holds about 22%. Higher is better. Always compare at the same list size.
Log loss and Brier score (KPI 1)	How close the percentages are to what happened. Confident wrong answers cost the most.	Percentages feed expected dollars and the what-if tab.	Lower is better. "Same rate for everyone" scores 0.185 log loss; the model 0.160.
AUC (KPI 1, ranking)	Pick one vendor that failed and one that did not: how often did the tool give the failing one the higher chance?	The list is only as good as the order.	0.5 is a coin toss, 1.0 is perfect. The model: 0.79.
Observed over forecast, calibration slope	Do 5% forecasts come true about 5% of the time?	A tool that is consistently too low or too high misleads the planning, even if its order is fine.	Observed over forecast of 1.0 is right on average; above 1 means forecasts ran too low, below 1 too high. A slope below 1 means forecasts are too extreme: the high ones too high, the low ones too low.
Typical dollar miss (KPI 2)	On average, how many dollars the cost estimate is off for a failure that actually happened.	Dollars set the cost of a failure for the what-if tab.	Compare with using the plain average.
Interval coverage	The 80% range should contain what happens about 80% of the time.	It says how far to trust the range shown next to each number.	Well below 80% means the ranges are too narrow.

3. KPIs, baselines and verdicts (1d)

We kept the Week 10 KPIs. KPI 1 is the chance of a material failure next week for each vendor. KPI 2 is the dollar cost given that a failure happens. The decision-level result combines them: the weekly list is the top k vendors by chance times cost, and because every vendor in a given week shares the same cost multiplier, that is the same list as ranking by chance alone. Baselines come in two kinds: probability baselines (the same rate for every vendor, the category rate, the vendor's own smoothed history, a gradient-boosted benchmark, and the model without its rejection inputs) and ranking rules a manager could apply without a model (delivery volume, trailing 13-week rejection rate, and today's on-time and fill-rate rules with a "loud recent failure jumps the queue" step).

KPI 1: chance of a failure next week

Method	AUC	Brier	Log loss	Slope	2026 AUC	2026 log loss	2026 slope
This model	0.789	0.0412	0.1599	1.03	0.749	0.1944	0.76
Vendor's own history	0.785	0.0414	0.1619	0.88	0.745	0.1974	0.66
Category failure rate	0.769	0.0417	0.1623	1.01	0.740	0.1944	0.78
Same rate for every vendor	0.487	0.0435	0.1854	—	0.500	0.2161	—
Gradient-boosted trees	0.772	0.0419	0.1634	0.94	0.742	0.1961	0.76
Model without rejection rates	0.788	0.0413	0.1602	—	0.754	0.1934	—

Development columns: 4,846 rolling out-of-fold vendor-weeks, 221 failures. 2026: 2,441 vendor-weeks, 136 failures. Slope is the calibration slope (1.0 is ideal). "Same rate for every vendor" has no ranking ability by construction.

Model minus baseline, with paired 95% intervals from a bootstrap over whole weeks. For log loss a negative number favours the model; for AUC a positive number does.

Against	Log loss, 2024-25	AUC, 2024-25	Log loss, 2026	AUC, 2026
Vendor's own history	−0.0020 [−0.0042, +0.0001]	+0.004 [−0.005, +0.011]	−0.0030 [−0.0067, +0.0004]	+0.004 [−0.009, +0.018]
Category failure rate	−0.0023 [−0.0042, −0.0005]	+0.020 [+0.009, +0.031]	0.0000 [−0.0033, +0.0033]	+0.010 [−0.008, +0.026]
Same rate for every vendor	−0.0255 [−0.0316, −0.0188]	+0.302 [+0.255, +0.344]	−0.0217 [−0.0296, −0.0137]	+0.249 [+0.214, +0.285]
Gradient-boosted trees	−0.0034 [−0.0057, −0.0010]	+0.017 [+0.003, +0.029]	−0.0018 [−0.0054, +0.0017]	+0.007 [−0.010, +0.027]
Model without rejection rates	−0.0003 [−0.0012, +0.0007]	0.000 [−0.004, +0.004]	+0.0009 [−0.0009, +0.0024]	−0.004 [−0.010, +0.002]

Verdict: not a meaningful improvement by the rule. The model beats the category rate on log loss (−0.0023 [−0.0042, −0.0005]) and is better than the vendor's own history by a margin whose interval includes zero (−0.0020 [−0.0042, +0.0001]). It beat the better of the two simple baselines in 5 of 8 blocks, short of the 6 required, and its AUC is only +0.004 above the vendor's own history (rule: 0.03). The rejection inputs add almost nothing: the model without them is statistically the same (−0.0003 [−0.0012, +0.0007]). In plain terms the forecasts are good and well calibrated, but most of their skill is the category and the vendor's own record, which a simple table also captures. In 2026 the model ties the category rate on log loss (0.0000 [−0.0033, +0.0033]), the same direction, so we call the pattern consistent.

KPI 2: dollar cost when a failure happens

Method	Typical miss	Bias	80% range holds	2026 miss	2026 bias	2026 holds
Cost model: next week's occupancy and banquet (used in the app)	$1,163	0.95	74%	$1,221	1.13	77%
Cost model, strict: last week's values only	$1,172	0.98	79%	$1,274	1.08	76%
Average cost by category	$1,166	0.97	77%	$1,254	1.12	79%
Plain average of past failure costs	$1,170	0.97	76%	$1,274	1.10	78%

219 failing vendor-weeks in the 2024-25 tests (mean $1,847, median $1,384, 10th to 90th percentile $444 to $3,696) and 138 in 2026 (mean $2,083). Bias is actual total divided by predicted total (1.00 is unbiased).

Verdict: not a meaningful improvement. The cost model's typical miss is 0.6% below the plain average (difference −8 dollars, 95% interval −65 to +54); the rule asked for 5%. Failure costs vary widely and the occupancy and banquet inputs do not explain the variation. The practical reading: treat the dollar figure as "a failure here costs about this much, give or take a lot", and rely on the range shown rather than the point value.

Decision level: the weekly review list

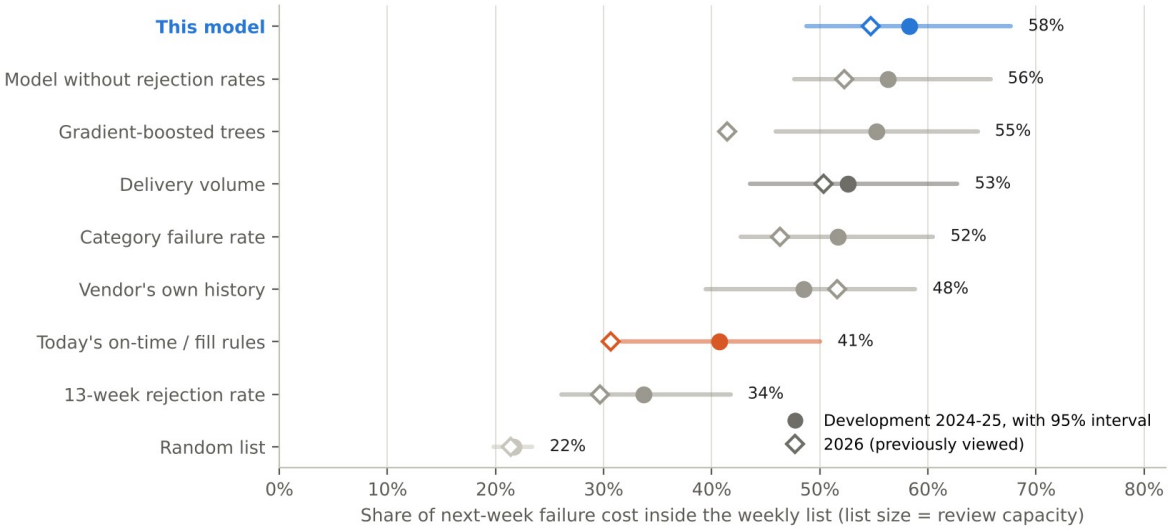


Figure 1. Share of next-week failure cost inside the weekly list, list size equal to the week's review capacity (mean 9.7 vendors of about 44 eligible). Filled circles with intervals: 2024-25 tests. Open diamonds: 2026.

List built by	Cost held, 2024-25	Model minus this, 2024-25	Cost held, 2026	Model minus this, 2026
This model	58.3%		54.7%	
Model without rejection rates	56.3%	+2.0 [−2.8, +7.6]	52.2%	+2.5 [−1.6, +10.2]
Gradient-boosted trees	55.2%	+3.1 [−5.4, +11.0]	41.4%	+13.3 [+4.5, +22.7]
Delivery volume	52.6%	+5.7 [−3.0, +14.0]	50.4%	+4.3 [−2.6, +12.1]
Category failure rate	51.7%	+6.6 [+1.3, +13.1]	46.3%	+8.4 [+1.0, +16.4]
Vendor's own history	48.5%	+9.8 [+3.7, +17.0]	51.6%	+3.1 [−4.3, +10.7]
Today's on-time / fill rules	40.7%	+17.6 [+6.9, +28.6]	30.6%	+24.1 [+10.7, +36.2]
13-week rejection rate	33.7%	+24.6 [+15.9, +33.0]	29.7%	+25.1 [+9.1, +40.1]
Random list	21.7%	+36.6 [+27.6, +45.4]	21.4%	+33.3 [+23.5, +43.6]

Differences are in percentage points with a paired 95% interval from a bootstrap over weeks. Failures caught (the count version of the same measure): model 57% in development and 51% in 2026, today's rules 42% and 32%.

Verdict. Against today's rules the model is clearly better in both periods (+17.6 [+6.9, +28.6] and +24.1 [+10.7, +36.2]). Against the strongest simple list, delivery volume, the difference is +5.7 [−3.0, +14.0] in development and +4.3 [−2.6, +12.1] in 2026; the point estimate is near the 5-point threshold but the interval includes zero, so the rule is not met. The model's list held more cost than the volume list in 6 of 8 blocks. We cannot say that the model finds something volume does not; we can say it matches volume and beats everything the manager uses today.

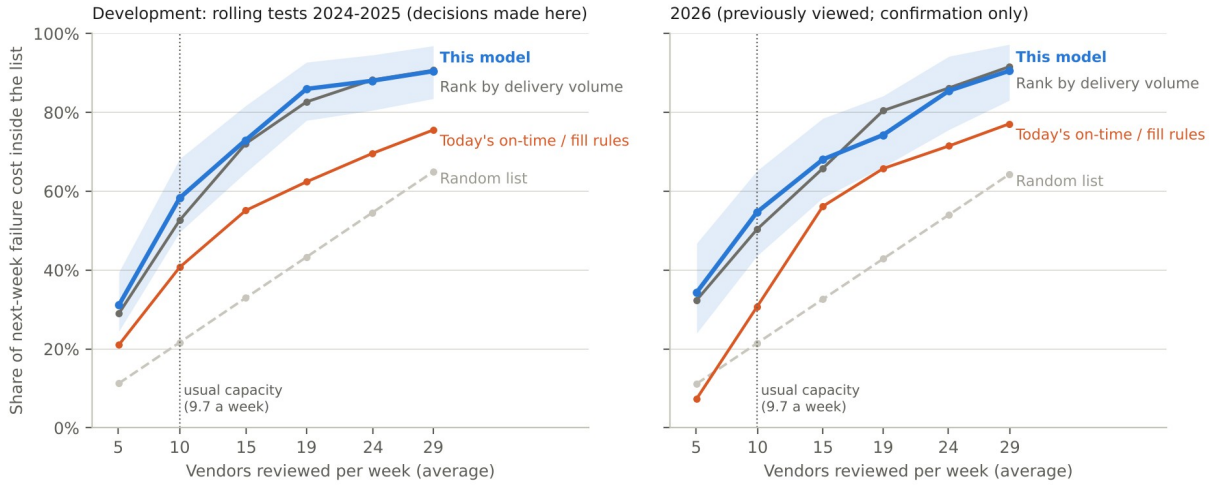


Figure 2. What a bigger list buys. Lines are the average share of next-week failure cost inside the list when the list is 0.5, 1, 1.5, 2, 2.5 and 3 times the review capacity. Band: 95% interval for the model.

List size (vendors a week)	Model	Volume	Today's rules	Random	2026 model	2026 volume
0.5× capacity (5.0)	31%	29%	21%	11%	34%	32%
1× capacity (9.7)	58%	53%	41%	22%	55%	50%
1.5× capacity (14.8)	73%	72%	55%	33%	68%	66%
2× capacity (19.4)	86%	83%	62%	43%	74%	80%
2.5× capacity (24.5)	88%	88%	70%	55%	85%	86%
3× capacity (29.1)	90%	91%	76%	65%	91%	92%

Left block: 2024-25 tests. The last two columns repeat the model and volume lists for 2026. The model and the volume list meet from about 2.5 times capacity upward.

On a fixed target such as 80%. At the usual capacity the list holds about half of the cost, so a target of 80% is not reached by the weekly list. Reaching about 80% takes roughly twice to two and a half times the capacity (about 19 to 24 reviews a week, around half of the eligible vendors): 86% at 2× in development and 74% (2×) to 85% (2.5×) in 2026. The right comparison for 55% to 58% is the random list at the same size (about 22%) and today's rules (31% to 41%), not a fixed 80%.

4. Where the tool does well and badly (1e)

We cut the rolling out-of-fold predictions (2024 to 2025) by vendor category, season, vendor size, delivery volume, open corrective action, length of recent history, recent failure, recent rejections and banquet weeks. For each group we compared failures that happened with failures forecast (1.0 is right on average) and measured how many of the group's failures were on the weekly list.

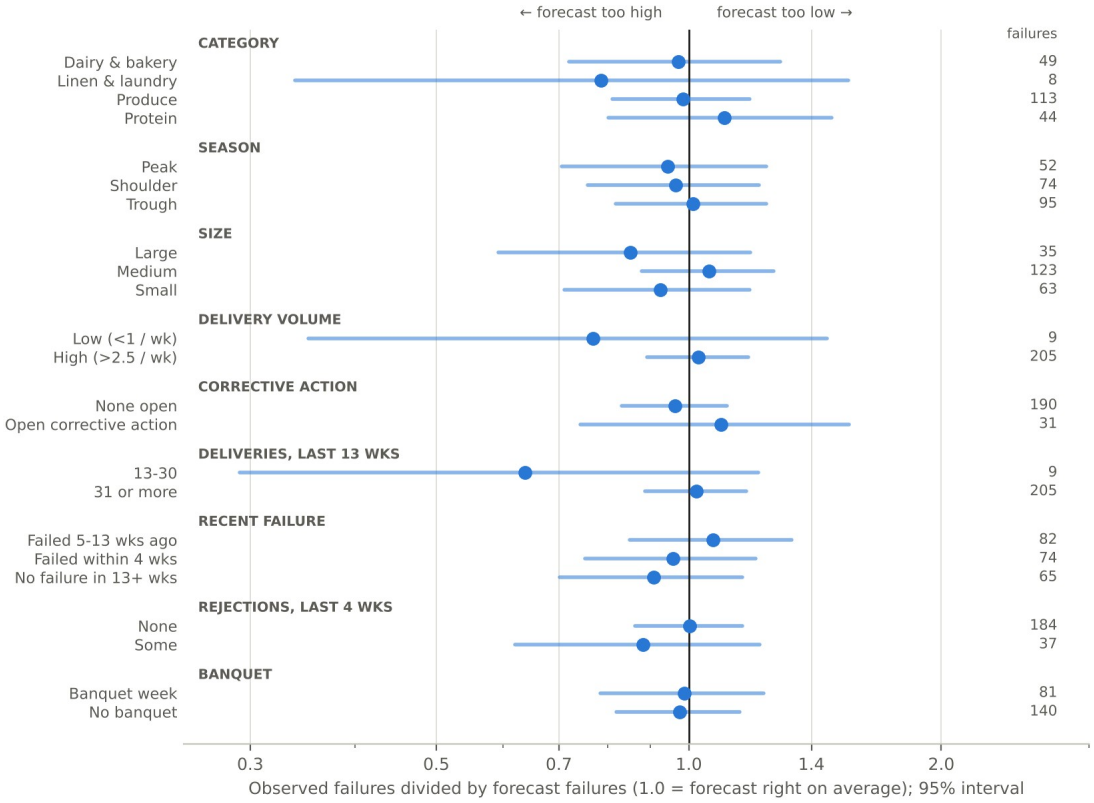


Figure 3. Observed failures divided by forecast failures, 2024-25 tests, groups with at least 8 failures. Every interval includes 1.0. Wide bars are small groups. Groups with fewer than 8 failures are not drawn because they cannot be judged: guest amenities (3), MRO (0), non-alcoholic beverage (4), mid-volume vendors (7) and the two shortest histories (2 and 5).

Group	Failures	Observed ÷ forecast	Share of its failures on the list	2026: observed ÷ forecast	2026: share on the list
Produce	113	0.98	78%	1.15	77%
Protein	44	1.10	11%	1.73	21%
Dairy and bakery	49	0.97	33%	0.79	31%
Peak weeks	52	0.94	33%	1.17	21%
Shoulder weeks	74	0.97	54%	1.35	52%
Trough weeks	95	1.01	55%	1.24	55%
Large vendors	35	0.85	26%	1.61	33%
Small vendors	63	0.92	63%	1.18	60%
Open corrective action	31	1.09	0%	0.87	0%
Failed 5 to 13 weeks ago	82	1.07	74%	1.43	57%
No failure in 13+ weeks	65	0.91	26%	1.54	27%
Rejections in last 4 weeks	37	0.88	43%	0.85	46%

Group	Failures	Observed ÷ forecast	Share of its failures on the list	2026: observed ÷ forecast	2026: share on the list
Banquet week	81	0.99	46%	1.18	30%

Overall in 2024-25 the list holds 49% of all failures, or 57% of failures at vendors eligible for review. Vendors with an open corrective action are held out of the list by design, so 0% is expected for them.

What this means for someone using the tool

The percentages can be read at face value; the list cannot be read as complete. Observed over forecast is consistent with 1.0 in every group we could test, and across the eight quarters it stayed between 0.80 and 1.12. What varies is how much of a group's failure the list reaches.

Produce is where the list works; protein and dairy are where it does not. Produce suppliers account for 113 of the 221 failures and the list catches 78% of them. For protein it catches 11% of 44 and for dairy and bakery 33% of 49, although together they carry about 47% of the failure cost. A manager should keep a separate eye on protein and dairy suppliers. The 2026 table shows the same pattern.

Peak weeks are covered least. Review capacity is smallest when the team is busiest (a list of 5.7 vendors of 46 in peak weeks against 11.1 of 44 otherwise), so the list holds 33% of peak-week failures against 54% in other weeks. The ranking inside peak weeks is as good (AUC 0.76); it simply covers less.

Vendors under an open corrective action fail more often (12.1% of vendor-weeks against 4.1%) and are not on the list. That is intended, because the manager is already acting, but it means 31 failures in 2024 to 2025 occurred where the list was blind by design. The app shows these vendors in a separate "held out" line with their forecasts.

Vendors with little recent delivery activity almost never fail. Under 13 deliveries in the last 13 weeks, there were 7 failures in 1,727 vendor-weeks, so spending a review on them is rarely worthwhile; vendors with no deliveries at all are shown as "not scored" rather than as zero risk.

Drift in 2026 was largest in protein and large vendors. Failures ran above forecast in most groups, as the declared 2026 shift implies, and observed over forecast was 1.73 for protein (95% interval 1.23 to 2.37) and 1.61 for large vendors (1.11 to 2.27), against about 1.0 in development. That is the declared 2026 shift showing up, and it is why the app carries a drift monitor.

5. Improvements we tried (2a to 2c)

We chose each candidate because of something the error analysis showed, and judged every one the same way: the same eight rolling blocks, the same rows, the same seeds and the same paired bootstrap, on development data only. The rules were fixed before the experiments ran. A change to the model was kept only if log loss improved by at least 0.001, it did so in at least 6 of 8 blocks, and the share of failure cost inside the list was not more than 1 point lower. A change to recalibration followed the same rule without the list clause. A change to the cost ranking needed 1 point more cost in the list in at least 5 of 8 blocks. A change to the dollar estimate needed a typical miss at least 1% lower, in at least 6 of 8 blocks, with a paired 95% interval that excludes zero. With thirteen comparisons one can win by luck, which is why the block-consistency requirement is there.

Two of the thirteen came from Debra Senteu's screening script (code/15_select_improvements.py), which compared two ideas on two yearly folds (train 2023, test 2024; train 2023 to 2024, test 2025): treating delivery volume as a fixed multiplier on the failure chance (J), and adding vendor category to the dollar estimate (K). Her script judged them by whether the average was lower, and found no gain for J and a $13 lower typical miss for K ($1,161 against $1,174). That is a good screen but two folds and no interval cannot separate a 1% change from noise, so we ran both on the same eight blocks as everything else, with a stated rule. We wrote the rules for J and K knowing her result, so they are a fair reading, not a blind test. (Her script also hard-coded a vendor-pooling value of 1.0 where the fitted Week 10 model uses 0.01; we corrected that, and her decisions did not change.)

Candidate	Why we tried it	Log loss vs Week 10	Blocks better	Cost held in list vs Week 10 (points)	Decision
A1 Recency weighting, half-life 52 weeks	Observed over forecast moved between quarters; recent data might matter more	−0.0001	5/8	+1.3 [−0.5, +3.4]	Rejected
A2 Recency weighting, half-life 104 weeks	As A1, with a longer memory	−0.0001	5/8	−0.1 [−2.1, +1.8]	Rejected
B1 Smoothed rejection rates (prior = 5 deliveries)	Four-week rejection rates rest on few deliveries and swing	−0.0002	5/8	+0.5 [0.0, +1.7]	Rejected
B2 Smoothed rejection rates (prior = 15 deliveries)	As B1, with heavier smoothing	−0.0003	5/8	+0.1 [−1.2, +1.3]	Rejected
C Add failures in last 13 weeks	The list catches 74% of failures that follow a recent failure and 26% of those that do not	+0.0003	1/8	+0.7 [−0.5, +2.4]	Rejected
D1 Drop 13-week rejection rate	Rejection groups show little added signal	−0.0001	5/8	−0.2 [−1.9, +1.2]	Rejected
D2 Drop 4-week rejection rate	As D1, for the four-week rate	+0.0005	3/8	−2.0 [−7.1, +2.7]	Rejected
F Trailing recalibration (26-week observed/expected)	Quarterly drift in observed over forecast	+0.0002	1/8	0.0 [0.0, 0.0]	Rejected
G1 Weaker vendor effects (lam_v = 16)	Vendor-specific signal is weak; shrink vendors harder toward their category	+0.0004	2/8	+2.1 [−0.2, +5.5]	Rejected
G2 Stronger vendor effects (lam_v = 4)	The opposite check: let vendors differ more	−0.0002	5/8	−3.6 [−7.2, −0.3]	Rejected

Candidate	Why we tried it	Log loss vs Week 10	Blocks better	Cost held in list vs Week 10 (points)	Decision
J Fixed exposure offset (g = 1) on shrunk exposure	Debra's screen: volume as a fixed multiplier on the failure chance (as in the Week 9 model), with exposure shrunk toward the category mean	+0.0003	3/8	−0.4 [−4.3, +3.9]	Rejected
E. Category-specific failure cost (rank by chance × category cost)	Protein and dairy carry about half of the failure cost but little of it is caught	unchanged	1/8	−1.1 [−4.9, +1.2]	Rejected
K. Vendor category added to the dollar estimate	Debra's screen: typical miss $13 lower on two folds. Here: −$10 (−0.8%; interval −$37 to +$14). The right-hand column is the list effect of pricing by category	dollars only	5/8 dollars; 1/8 list	−2.3 [−6.2, +0.2]	Rejected

Baseline (the Week 10 model): log loss 0.1599, 58.3% of failure cost inside the list. Cost-ranking candidate E: cost does not differ by category (Kruskal-Wallis p = 0.40), so ranking by chance times category cost gives the same list as ranking by chance.

Result: the Week 10 model stays. None of the candidates reached the log-loss threshold with the required consistency. The closest, weaker vendor effects, raised cost in the list by about two points but made log loss slightly worse and was better in only 2 of 8 blocks. Debra's two ideas fare no better: the shrunk exposure offset is slightly worse on log loss and better in only 3 of 8 blocks, and adding category to the dollar estimate lowers the typical miss by $10 (0.8%) with an interval of −$37 to +$14, below the 1% bar and inconsistent across blocks (5 of 8), while pricing the list by category lowers the cost held in it. The pattern across all thirteen is the useful finding: the data contain only a modest amount of vendor-specific signal beyond category and volume, so more tuning does not help.

What we did change. First, a correctness fix in the cost estimate: the cost model was fitted on the occupancy and banquet flag of the week a failure happens, but the Week 10 app priced next week's failure with the selected week's own values. The app now feeds the cost model next week's expected occupancy and banquet flag, so the inputs at scoring match those at fitting. It is not an accuracy gain (the difference is noise). Second, the decision-support features in section 7. The probability model and its coefficients are unchanged, and a check that the Week 11 pipeline reproduces the Week 10 2026 predictions agrees to 9.9e-17.

6. Remaining limitations (2d)

Synthetic data.	Every number shows how the method behaves on a generated resort, including a declared 2026 shift. It does not say how a real property would perform.
2026 is a previously viewed holdout.	It cannot serve as a clean final test. We used it only for confirmation, but the team has seen the Week 10 results, and the "meaningful improvement" thresholds were written after those results were known.
The list holds about half the cost, not most of it.	At capacity the list holds 58% (2024 to 2025) and 55% (2026) of next-week failure cost. Reaching 80% takes about two to two and a half times the capacity.
No demonstrated edge over delivery volume.	The model matches ranking by volume within the noise of this data. It may do better with more weeks or richer inputs, but we cannot show it.
Weak vendor-specific signal.	Recent rejections add little to the ranking, and the model without them is statistically the same. Early deterioration of one vendor is hard to detect with three years of weekly data.
Dollar estimates are averages.	Failure cost varies from a few hundred to several thousand dollars and the inputs we have do not explain which failures are expensive.

Drift.	The intervals reflect uncertainty in the fitted coefficients, not a changed world, so they understate risk when conditions shift (weekly dollar intervals covered 71% in 2026, not 80%). The monitor flags this; it does not correct it.
Uneven coverage.	Protein, dairy, peak weeks and large vendors are covered least, and vendors under open corrective action are held out of the list by design.
Scenarios are model arithmetic.	The what-if tab shows what the fitted model would say if a record looked different. It does not show that changing an input would change what happens.
One outside test.	The structured test in section 8 involved one person. It shows what went wrong for that person, not how often.

7. Decision support in the revised app (3a to 3f)

What to tell the general manager (3a)

> "Each Monday the tool ranks about 44 vendors by the chance of a material failure next week. The vendors at the top (as few as four in the busiest weeks, up to twelve in quiet ones) held about half of next-week failure cost in testing, around two and a half times what a random list of that size would hold and 18 to 24 points more than the on-time and fill-rate rules we use today. It is not shown to beat simply reviewing our highest-volume vendors, and it cannot say which supplier will fail: a vendor at 19% still does not fail in four of five such weeks. Dollar figures are averages; a real failure cost between $444 and $3,696 in four cases out of five. It is weakest for protein and dairy suppliers and in peak weeks. All of this was measured on synthetic data."

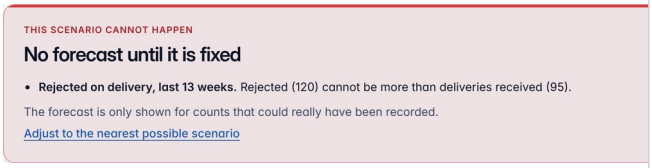
The app states the same limits on its Evaluation tab, in the manager's terms, with the baseline tables, the list-size chart, the segment table and the improvements table behind them.

Which inputs a user can change (3c)

Input	Label in the app	Why
Deliveries received, last 13 weeks and last 4 weeks	You can change	They are the record as counted on that Monday. A scenario means "if the record had looked like this".
Rejected on delivery, last 13 weeks and last 4 weeks	You can change	Same. A scenario here is not what a corrective action would achieve: rejections already happened.
Vendor category and size; the vendor's own 2023 to 2025 record	Fixed	Part of the vendor, learned from history; cannot be edited in a scenario.
Season, review capacity, next week's occupancy and banquet booking	Outside your control	Set by the calendar and the property.
Failure-rate shift for all vendors; cost of a failure	Outside your control	Scenario conditions (a drift like 2026, a price spike), not choices the manager makes.

Input checks and the invalid-scenario message (3d)

The what-if inputs are counts, not sliders: deliveries and rejections for the last 13 and last 4 weeks, each shown next to the value recorded for that Monday. The tool computes the rates from the counts, so combinations that could never be recorded cannot be entered. It checks, in this order: whole numbers of at least zero; rejected not above received in each window; the 4-week figures not above the 13-week figures (the last 4 weeks are part of the last 13); and weeks 5 to 13 (the 13-week figure minus the 4-week figure) not holding more rejections than deliveries. A vendor with no deliveries in 13 weeks cannot be scored and the tool never treats it as zero risk. Values that are possible but outside what the model has seen are allowed and flagged as extrapolation. When a scenario is impossible the tool shows no forecast, names every broken rule, and offers a button that moves to the nearest possible scenario.



Typing 120 rejected deliveries when 95 were received.

Automated checks (qa/wk11_whatif.js) cover decimals, negatives, empty boxes, each relationship above, the no-deliveries case, the repair button, quick scenarios, phone width and console errors: 32 checks, all passing on the delivered build.

Baseline against scenario, and the automatic summary (3e, 3f)

Changing any input fills a two-column table: the vendor as recorded on that Monday against the scenario, the inputs that changed (shaded), the model's answers with 80% intervals, the difference, and where the vendor would land on that week's list. Below it the app writes a summary from the inputs, in four parts: what changed, what the model says, trade-offs to weigh, and what the result does and does not tell you, including the line "Model-based, not proven cause".

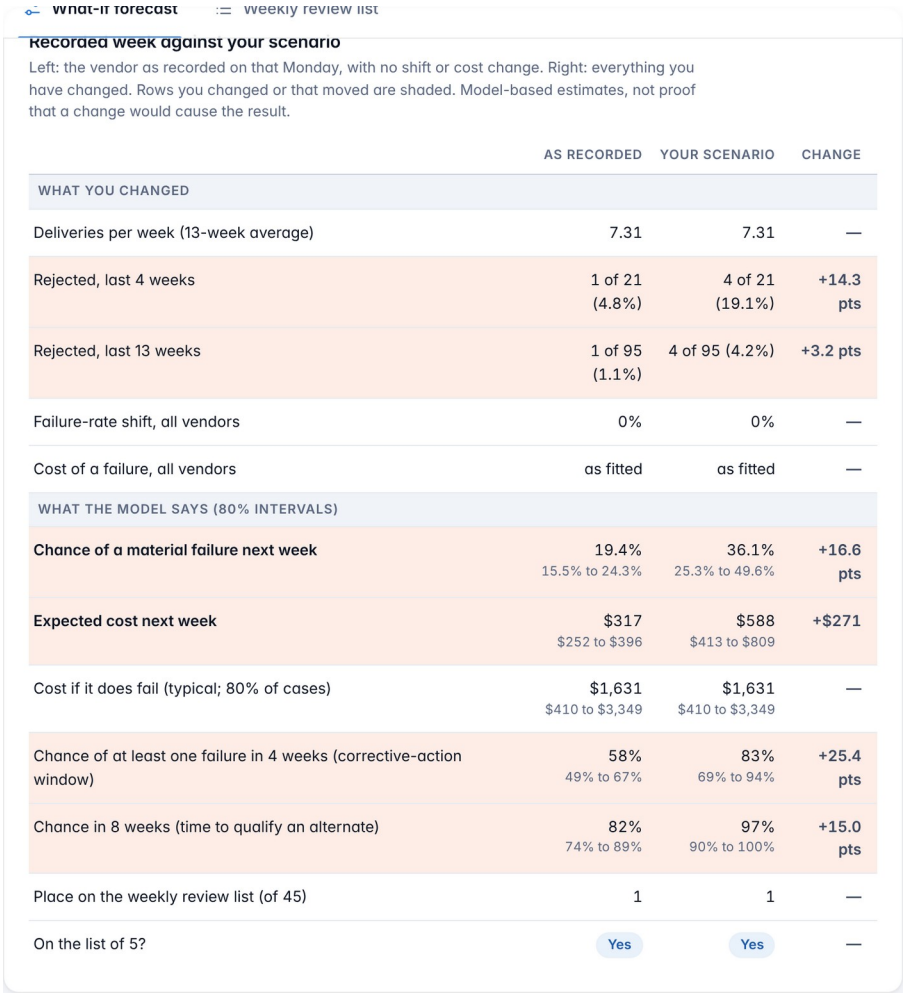


Figure 4. Produce 002, week 26: three more rejected deliveries than recorded. The chance rises from 19.4% to 36.1% and the vendor stays first on the list of five; a warning (not shown here) says the 4-week rejection rate is beyond what the model has seen.

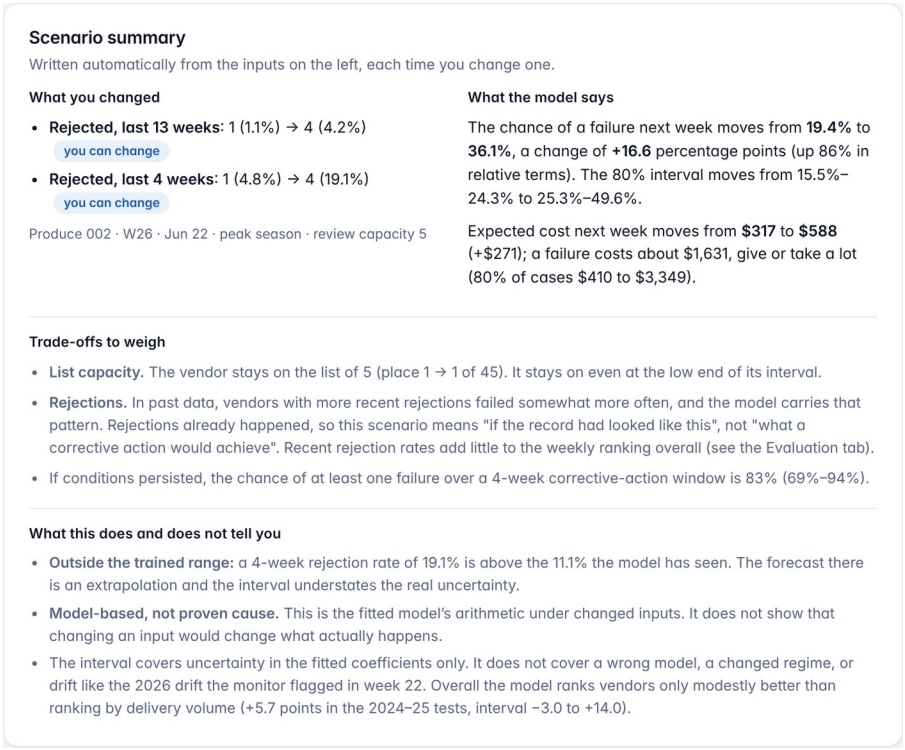


Figure 5. The automatic summary for the same scenario.

Explaining one prediction (3b)

Built in Week 10 and kept: the tool breaks any forecast into the contribution of each input, walking from a reference vendor-week to this one. Because the model is linear on its link scale, these are exact Shapley values, and they are separate from the overall importance chart on the Model performance tab.

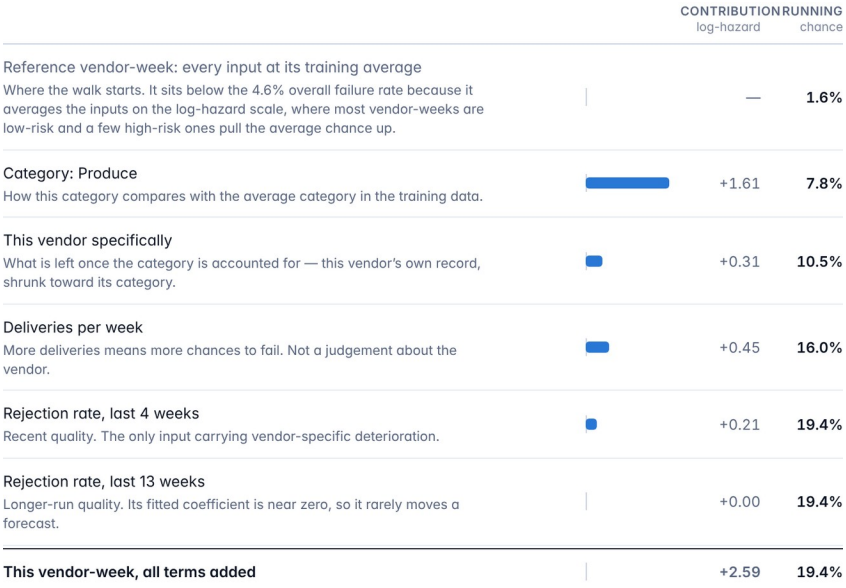


Figure 6. Produce 002, week 26: category +1.61, the vendor's own record +0.31, deliveries +0.45, four-week rejections +0.21, thirteen-week rejections about zero; the contributions add to the distance from the reference vendor-week, and the running chance ends at 19.4%.

8. Structured test with someone outside the group (4)

We prepared a kit (usability_test/Week11_Usability_Test_Kit.docx): a protocol with a neutral script and rules for hints, a one-page task sheet of eight tasks (read the weekly list, read a forecast and its interval, find why a vendor scores what it does, tell changeable from fixed inputs, run a what-if, recover from an impossible scenario, judge whether to rely on the tool, and use the summary), an observation form, post-test questions, a findings log and a facilitator answer key with the places we expected trouble, written before the session.

Status: the session has not been run, and this section holds no results. It will be completed from the real session record (who took part, what happened on each task, what we changed because of it). Nothing has been simulated or filled in.

9. The package: what is in it and how to run it (5)

Item	What it is
Week10_frozen/	The Week 10 submission, unchanged, so the earlier version stays identifiable.
app/index.html	The revised app: one self-contained file with four tabs (Model performance, Evaluation & limits, What-if forecast, Weekly review list). Open it in any browser. app/artifact_page.html is the same page for hosting; the Streamlit wrapper in streamlit/ embeds it.
data/source/ and data/	The shareable synthetic source tables and the derived tables, the fitted model, the Week 11 evaluation files (wk11_*.json, wk11_dev_oof.csv).
code/	Pipeline scripts. bash code/run_all.sh rebuilds everything in about three minutes: model table, fit, Debra's two-fold screen (15), Week 11 evaluation (20), error analysis (21), improvements (22), app data (12, 14) and app (13).
docs/	Logs written by the scripts (wk11_evaluation_log.txt, wk11_error_analysis.txt, wk11_improvements_log.txt), the changelog and the Week 10 documents.
report/	This report (.docx and .pdf), its figures and the scripts that build them.
usability_test/	The test kit and, after the session, the completed forms and findings log.
video/	The seven-minute video script and the silent screen capture to narrate over.
qa/	Browser tests with Playwright: Python-to-JavaScript parity, the what-if checks, and screenshots.
README.md, requirements.txt, requirements-tested.txt	Setup and dependencies: Python 3.10 or later with numpy, pandas, scipy and scikit-learn (tested versions are listed); Node with Playwright only for the optional browser tests.

Reproducibility. The Week 11 pipeline reproduces the Week 10 2026 predictions to within 9.9e-17. Some decision-list shares differ from the Week 10 report by up to one point because Week 11 averages over tie-break draws in the rule-based lists and uses the exact expectation for the random list; the conclusions do not change.

Contributions. See CONTRIBUTIONS.md in the package. It is a draft for the team to confirm; we have not written anything there that the team has not told us.

Methods references. Van Calster et al. (2019) BMC Medicine 17:230 on calibration; Hand (2009) Machine Learning 77:103 on the limits of AUC; Elkan (2001) IJCAI on cost-sensitive decisions; Davis et al. (2017) AMIA Annual Symposium on calibration drift; Efron and Morris (1977) on shrinkage; Goodwin, Önkal and Thomson (2010) EJOR 205:195 on prediction intervals in planning.